AL-2002: Artificial Intelligence Lab

Serial No:

Sessional Exam

Attempt Time: 1 Hour 30 Mins
Submission Time: 10 minutes
Total Time: 1 Hr. 40 mins.
Total Marks: 90

Friday, 3rd April, 2026

Course Instructors

Zill-E-Huma, Imran Khan, Ali Umer

Signature of Invigilator

Student Name _____ Roll No. _____ Section _____
Signature _____

Invigilator Instructions:

This is a **Close book paper**; students are not allowed to use the Internet or any Gadgets including a USB device. **Note their submission time on paper**. No student is allowed to leave the exam before the submission time unless they choose to do so without submitting their exam. In such cases, their exam will be considered as missed, and this should be clearly noted on their paper.

DO NOT OPEN THE QUESTION BOOK OR START UNTIL INSTRUCTED.

Instructions:

1. Attempt all of them. Read the question carefully and then attempt it.
2. If you need to specify anything, just write it as a **multi-line comment** in your code file.
3. After being asked to commence the exam, please verify that you have **2** different pages including this title page. There is total of **3** questions.
4. Sharing anything is **not** allowed.
5. Understanding of the question is part of the exam, attempt at yourself whatever you understand or suppose it if anything is necessary or missing and mention your assumption in comment.
6. Cutting and overwriting is **not** allowed.
7. Use of internet is not allowed.
8. If plagiarism or AI generated content is found in your code, **an F grade** will be awarded.
9. If the coding file is not named exactly as mentioned (**StudentID_StudentName_Section**), 10 marks will be deducted.

	Q-1	Q-2	Q3	Total
Obtained Marks				
Total Marks	**35**	**20**	**35**	**90**

Question 1

Part A:

In the classic game of Tic-Tac-Toe, two players take turns marking spaces on a 3x3 grid with their respective symbols, typically 'X' and 'O'. The objective is to achieve a row, column, or diagonal of three of their symbols in a row before the opponent does. The game ends in a draw if the grid is filled without a winner.

Design a local search algorithm to optimize the strategy for the 'X' player in Tic-Tac-Toe, aiming to increase the likelihood of winning or achieving a draw against an opponent following a simple rule-based strategy.

Objective: Develop a local search algorithm that iteratively refines the 'X' player's moves based on the current game state. The algorithm should explore neighboring states by considering possible moves and selecting the one that maximizes the chances of winning or preventing the opponent from winning.

Requirements:

1. The local search algorithm should start with an initial move strategy and iteratively explore nearby moves to improve its chances of winning.

2. The optimization process should consider the opponent's likely responses and adjust the strategy accordingly.

3. The algorithm should be simple and computationally efficient, suitable for real-time decision-making in a game scenario.

Part B: Solve it on paper

1.
```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(arr.mean(axis=1))
```
A. [2, 5, 8]
B. [3, 4, 5]
C. [6, 15, 24]
D. [1, 2, 3]

2.
```
import pandas as pd
data = {'A': [1, 2, 3], 'B': [4, 5, 6]}
df = pd.DataFrame(data)
print(df.loc[0])
```
A. A 1 B 4
B. A 1 B 2
C. A 4 B 5
D. A 1 B 4

3.
```
class ModelBasedAgent:
```

```
def __init__(self):
  self.state = {'A': 'Action1', 'B': 'Action2'}
def act(self, percept):
    if percept in self.state:
      return self.state[percept] else:
       return 'Action3'
agent = ModelBasedAgent()
print(agent.act('B'))
```

A. Action1
B. Action2
C. Action3
D. Error: 'percept' is not defined

4.
```
import re
text = "hello 123 world" pattern =
r'\d+'
result = re.findall(pattern, text)
print(result)
```

A. ['123']
B. ['hello', 'world']
C. [123]
D. ['hello', '123', 'world']

5. What is the output of the following Python code snippet?

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

result = arr[[True, False, True, False,
True]] print(result)
```

A. [1 2 3]
B. [1 3 5]
C. [False True False True False]
D. Error

Question 2:

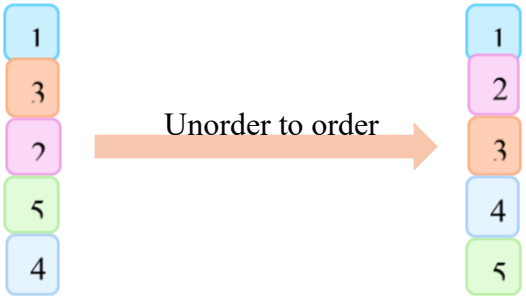
First, you need to design logic, like brainstorming or pseudo code, and then implement it. Brainstorming carries 5 marks.

Blocks World Problem:

Scenario: The blocks world problem involves organizing a set of numbered blocks to form a tower according to a predefined order. Blocks can either be stacked on top of each other or placed on the table, and only one block can be moved at a time. Each block has a specific

number, and they are initially stacked in a disorderly manner on a table. The objective is to go from an initial disordered state to a specified goal state using the minimum number of moves. Various heuristics, such as counting the number of incorrectly placed blocks or considering the entire stack structure, can be used to guide an A* search algorithm in finding the most efficient solution.

Example for understanding



Approach:

- **Heuristic Evaluation:** Develop a heuristic function that This heuristic calculates the number of blocks that are incorrectly placed on top of other blocks. For each block, it counts the number of blocks above it that should be below it in the final arrangement.
- **Search Strategy**: Design the A* search algorithm to intelligently explore the solution space by prioritizing states to find the sequence of moves to transition from the initial state to the goal state while minimizing the total cost.
- **Performance Evaluation**: Performance evaluation of the A* search algorithm for the block's world problem can be assessed based on several key factors: Evaluate the performance of the algorithm based on any metrics that is up to you choose you criteria or design your own criteria. Some examples are heuristic accuracy, optimality.

Explore the solution space by expanding nodes with lower total costs, ultimately finding the optimal or near-optimal solution to the problem.

Basic Structure: (Optional you can design your own)

```
class BlocksWorldSolver:
    def __init__(self, initial_state, goal_state):
        self.initial_state = initial_state
        self.goal_state = goal_state

    def is_goal_state(self, state):
        # Implementation of the check for the goal state

    def evaluate_state(self, state):
        # Implementation of the A* heuristic function
```



```
    def a_star_search(self):
        # Implementation of the A* search algorithm

# Example usage:
initial_state = [...]  # Define initial state
goal_state = [...]     # Define goal state
solver = BlocksWorldSolver(initial_state, goal_state)
solution = solver.a_star_search()
print(solution)
```

Question 3:

First, you need to design logic, like brainstorming, and then implement it. Brainstorming carries 3 marks.

Scenario

On a stormy autumn night in 2026, five tech-savvy entrepreneurs reunite in a hidden VR lounge, echoing the 1900 wager inspired by Phileas Fogg's *Around the World in 80 Days*. Decades ago, as university mates, they celebrated Fogg's feat with a simple trip to Paris's Le Procope. Now, at the dawn of AI-driven gaming, they stake $1 million on who can conquer North America's rails fastest - using an online console game built in Python. But with a twist: the game runs autonomous competing in simulated turns to claim routes, draw cards, and complete destinations. You must design this game to determine the ultimate winner. Your task is to **design and implement this GA-driven Ticket to Ride simulation**.

Problem Overview

Ticket to Ride is a strategic board game where players claim railway routes on a North American map using colored train cars, complete secret Destination Tickets, and build the longest continuous path for bonus points. Your task: Create a fully playable game in Python (text-based, multiplayer via console input or networked) but replace human players with 5 AI agents using Genetic Algorithms. Each AI evolves strategies over generations to maximize scores, simulating the high-stakes race. The game must handle setup (board), turns, scoring, and endgame faithfully, outputting a champion after 100+ simulated games.

What You Need to Build (Core Requirements)

- **Map Representation**: A graph of ~45 cities and ~80 routes (e.g., Montreal-Toronto: gray, length 5; Montreal-New York: blue, length 3). Use adjacency lists or dicts with route details (cities, length, color).
  Example

```
# Complete Ticket to Ride: North America Map (45 cities, ~80 routes)

map_graph = {

    "Montreal": [("Toronto", 3, "gray"), ("Boston", 2, "purple"), ("New York", 3, "blue")],
```



```
    "Toronto": [("Montreal", 3, "gray"), ("Chicago", 4, "orange"), ("Pittsburgh", 2, "green"),
("Sault St. Marie", 5, "orange")],

    "Boston": [("Montreal", 2, "purple"), ("New York", 2, "purple")],

    "New York": [("Montreal", 3, "blue"), ("Boston", 2, "purple"), ("Washington", 2, "white"),
("Philadelphia", 2, "yellow")],

    "Philadelphia": [("New York", 2, "yellow"), ("Washington", 2, "green")],

    "Washington": [("New York", 2, "white"), ("Raleigh", 2, "purple"), ("Philadelphia", 2,
"green")],

    "Raleigh": [("Washington", 2, "purple"), ("Atlanta", 2, "green")],

    "Atlanta": [("Raleigh", 2, "green"), ("New Orleans", 4, "blue"), ("Miami", 5, "yellow")],

    "Miami": [("Atlanta", 5, "yellow")],

    "New Orleans": [("Atlanta", 4, "blue"), ("Houston", 2, "orange")],

    "Houston": [("New Orleans", 2, "orange"), ("Dallas", 1, "red")],

    "Dallas": [("Houston", 1, "red"), ("Oklahoma City", 2, "yellow"), ("El Paso", 4, "orange")],

    "Oklahoma City": [("Dallas", 2, "yellow"), ("Kansas City", 2, "green"), ("Santa Fe", 5,
"gray")],

    "Kansas City": [("Oklahoma City", 2, "green"), ("St. Louis", 2, "yellow"), ("Denver", 4,
"blue")],

    "St. Louis": [("Kansas City", 2, "yellow"), ("Chicago", 2, "black"), ("Oklahoma City", 3,
"red"), ("Nashville", 2, "orange")],

    "Nashville": [("St. Louis", 2, "orange"), ("Atlanta", 3, "red")],

    "Chicago": [("Toronto", 4, "orange"), ("St. Louis", 2, "black"), ("Omaha", 4, "blue"),
("Duluth", 3, "yellow")],

    "Omaha": [("Chicago", 4, "blue"), ("Denver", 4, "yellow"), ("Duluth", 4, "gray")],

    "Denver": [("Omaha", 4, "yellow"), ("Kansas City", 4, "blue"), ("Salt Lake City", 3, "red"),
("Santa Fe", 3, "purple")],

    "Santa Fe": [("Denver", 3, "purple"), ("Oklahoma City", 5, "gray"), ("Phoenix", 3, "green")],

    "Salt Lake City": [("Denver", 3, "red"), ("Las Vegas", 3, "orange"), ("San Francisco", 7,
"green")],

    "Las Vegas": [("Salt Lake City", 3, "orange"), ("Los Angeles", 2, "yellow")],

    "San Francisco": [("Salt Lake City", 7, "green"), ("Los Angeles", 3, "blue"), ("Portland", 5,
"gray")],
```

```
    "Los Angeles": [("San Francisco", 3, "blue"), ("Las Vegas", 2, "yellow"), ("Phoenix", 3, "red")],

    "Phoenix": [("Los Angeles", 3, "red"), ("Santa Fe", 3, "green"), ("El Paso", 3, "yellow")],

    "El Paso": [("Phoenix", 3, "yellow"), ("Dallas", 4, "orange"), ("Oklahoma City", 4, "blue")],

    "Portland": [("San Francisco", 5, "gray"), ("Seattle", 3, "blue")],

    "Seattle": [("Portland", 3, "blue"), ("Vancouver", 1, "green")],

    "Vancouver": [("Seattle", 1, "green"), ("Calgary", 4, "red")],

    "Calgary": [("Vancouver", 4, "red"), ("Winnipeg", 6, "yellow")],

    "Winnipeg": [("Calgary", 6, "yellow"), ("Duluth", 4, "red"), ("Toronto", 6, "green")],

    "Duluth": [("Winnipeg", 4, "red"), ("Chicago", 3, "yellow"), ("Sault St. Marie", 2, "purple"), ("Omaha", 4, "gray")],

    "Sault St. Marie": [("Duluth", 2, "purple"), ("Toronto", 5, "orange")],

    "El Paso": [("Dallas", 4, "orange"), ("Phoenix", 3, "yellow")],

    "Miami": [("Atlanta", 5, "yellow")],

    "Philadelphia": [("New York", 2, "yellow"), ("Washington", 2, "green")],

    "Nashville": [("Atlanta", 3, "red"), ("St. Louis", 2, "orange")],

    "San Francisco": [("Portland", 5, "gray"), ("Los Angeles", 3, "blue"), ("Salt Lake City", 7, "green")],

    "Los Angeles": [("San Francisco", 3, "blue"), ("Las Vegas", 2, "yellow"), ("Phoenix", 3, "red")],

    "Denver": [("Santa Fe", 3, "purple"), ("Kansas City", 4, "blue"), ("Salt Lake City", 3, "red")],

    "Santa Fe": [("Denver", 3, "purple"), ("Phoenix", 3, "green"), ("Oklahoma City", 5, "gray")],

    "Salt Lake City": [("Denver", 3, "red"), ("San Francisco", 7, "green"), ("Las Vegas", 3, "orange")],

    "El Paso": [("Phoenix", 3, "yellow"), ("Dallas", 4, "orange")]

}
```

- **Components**:

Component	Quantity/Details
Train Car Cards	110 cards (12-14 each of 8 colors + 14 Locomotives); deck with face-up pile (5 cards).
Destination Tickets	30 cards (e.g., "Sault St. Marie - Oklahoma City: 9 pts"); players keep 2-3 secretly.

Player Trains	45 per player (5 colors: Blue, Red, Green, Yellow, Black).
Scoring Track	0-200+ points; Longest Path bonus: 10 pts.

- **GA Integration**: 5 AI populations each with a different color(one per player color i.e. blue, red, green, yellow, black). Each AI player (Blue, Red, Green, Yellow, Black) will **not have just one strategy**, but many **possible strategies** at the same time. Example

```
Blue AI population → 50 individuals

Red AI population → 50 individuals

Green AI population → 50 individuals

Yellow AI population → 50 individuals

Black AI population → 50 individuals
```

  These strategies are represented as **individuals** (genomes) in a **population**.
  - Each **individual** is one possible way the AI can make decisions in the game (e.g., genes for: draw probability 0-1, route claim aggressiveness 0-1, ticket draw risk 0-1, path-building heuristic weights). Evolve over 50-100 generations using fitness score. Pick one individual from each population and continue the game and assign fitness.
  - **Selection**: Keep the best individuals.
  - **Crossover**: Combine pairs of selected individuals to produce children.
  - **Mutation**: Slightly modify children to explore new strategies.
  - **Next generation**: Replace old population with new children.

Repeat 50–100 generations so AI strategies evolve to maximize score.

Game Rules

Play proceeds clockwise; each turn, one action only:

1. **Draw Train Car Cards** (2 cards max): From 5 face-up or blind deck draw. Locomotives are wild; if 3+ face-up Locomotives, discard and refresh.
2. **Claim a Route**: Play matching color/length cards (e.g., 4 blue for blue route of 4). Place trains; discard cards. Gray routes: any color. Double routes: one per pair (2-3 players: block second). Max 1 route/turn; no must-connect rule.
3. **Draw Destination Tickets** (3 cards): Keep ≥1; return others to bottom.

Route Scoring Table (Hardcode this):

Route Length	Points
1	1
2	2
3	4

4	7
5	10
6	15

Win Conditions & Endgame

- Game ends when any player has ≤2 trains left (all get 1 final turn).
- **Final Score Calculation**:
    1. Sum route points (already tracked).
    2. Reveal Destinations: +points if connected via your routes; -points if not.
    3. Longest Continuous Path (same-color trains, loops OK, no reuse): +10 pts (ties all win).
- Winner: Highest score. Ties broken by most completed Destinations, then Longest Path.
- GA Winner: After 100 runs, the AI population with highest average score claims the $1M (output stats: best genome, avg score, evolution graph).

Scoring Scheme for Your Python Game

- **AI Fitness**:
    - Primary: total points.
    - Secondary: +bonus for Destinations completed (x2 weight), -penalty for unconnected tickets (x1.5).
    - Tertiary: path length.
- **Game Outputs**: Console logs turn (e.g., "Blue AI claims Vancouver-Seattle (green,4) for 7 pts"), final leaderboard, GA evolution summary (e.g., "Generation 50: Best fitness 210 pts").
- **Edge Cases**: Deck exhaustion (reshuffle discards); no cards left (skip draws); visualize map/routes claimed.